

INTRODUCTION TO MACHINE LEARNING AND NEURAL NETWORKS

LEAP Summer Bootcamp | Day 1



LEAP

LEARNING THE EARTH WITH
ARTIFICIAL INTELLIGENCE & PHYSICS



COLUMBIA | NYU | UC IRVINE | U. MINNESOTA | TEACHERS COLLEGE | NCAR | NASA | CarbonPlan

Schedule for the 2nd week of the Bootcamp



Gabriele Accarino

Columbia/LEAP

Day 1 & Day 2

Introduction to ML + ANNs



Matthew Blanke

NYU/LEAP

Day 3

GenAI



Viviana Acquaviva

CUNY/Columbia/LEAP

Day 4

LLMs 4 Science



LEAP

...Just a little bit more about me



Gabriele Accarino

Postdoctoral Research Scientist

Department of Earth and Environmental Engineering

Columbia University & LEAP NSF STC, New York, US

ga2673@columbia.edu

From Lecce (Italy)

Also called “The Florence of the South”



Background: Computer Engineering + Env. Science

Research Interests

- ML emulators of physical systems
- ML for extreme weather events detection
- GenAI



LEAP

Today's Agenda

- **What is Machine Learning? Definitions, types, and examples**
- **Applications in Climate & Earth Science**
- **Linear and Polynomial Regression**
- **Cost function, Gradient Descent, Bias-Variance tradeoff**
- **Perceptron and Artificial Neural Networks**
- **PyTorch Framework**



LEAP

Machine Learning Fundamentals

Definitions, types, examples, and pitfalls

Traditional Programming vs. ML

Traditional Programming

Data + Program



Computer



Output

Machine Learning

Data + Output



Computer



Program (Model)

Ref: Mitchell, T. (1997). Machine Learning. McGraw Hill.



LEAP

What is Machine Learning?

The goal of machine learning is to extract useful information from data and learn patterns that generalize to new, unseen examples.

Key idea

- Given data samples (inputs x , outputs y), learn a model that maps x to y
- The model is an input-output relationship learned from data
- Even simple methods like linear regression are part of ML methods
- x and y can be scalars, vectors, images, time series, or any combination



Supervised Learning

Learn from labeled data: each input x comes with a known output (label) y .

Regression

- Output is continuous (real-valued)
- Predict precipitation amounts
- Estimate sea surface temperature
- Forecast wind speed

Classification

- Output is discrete (categories)
- Cloud type: convective vs. stratiform?
- Is there a hurricane in this image?
- Anomaly: severe weather event?

Both require labeled datasets: input features paired with known correct outputs.



Other Types of Machine Learning

Unsupervised Learning

No labels. Learn structure, clusters, or compressed representations from data alone.

Examples:

- K-means clustering
- PCA for dimensionality reduction
- Anomaly detection

Reinforcement Learning

Learn through trial and error. Agent takes actions, receives rewards.

Examples:

- Optimizing renewable energy dispatch
- Adaptive weather forecast strategies
- Game playing (AlphaGo)

Self-Supervised Learning

Create labels from the data itself. Bridge between unsupervised and supervised.

Examples:

- Predicting masked regions of satellite images
- Contrastive learning on climate fields
- Foundation models



ML Applications in Climate & Earth Science

- **Prediction tasks**
 - Predict physical fields, e.g., temperature, precipitation
- **Classification tasks**
 - Distinguish cloud types (stratiform vs. convective), detect Tropical Cyclones in climate maps
- **Object detection**
 - Locate Tropical Cyclone center
- **Anomaly detection**
 - Severe weather, heat waves
- **Subgrid-scale parameterization**
 - Replace computationally heavy or uncertain parameterizations in numerical models with ML



Regression

Linear regression, cost function, gradient descent, and the bias-variance trade-off



Linear Regression

The simplest ML model: fit a straight line to data.

Model

$$\hat{y} = h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_{n-1} x_{n-1}$$

- The linear model is often called **hypothesis function** since this represents our hypothesis of the underlying relationship among data
- θ_n = weight vector (parameters to learn)
- x_n = input features/variables
- $\hat{y}$ = predicted output (e.g., precipitation amount)
- **Goal:** find weights θ that minimize prediction errors on training data
- **Important:** the model is linear in the parameters! It might be non-linear in the input

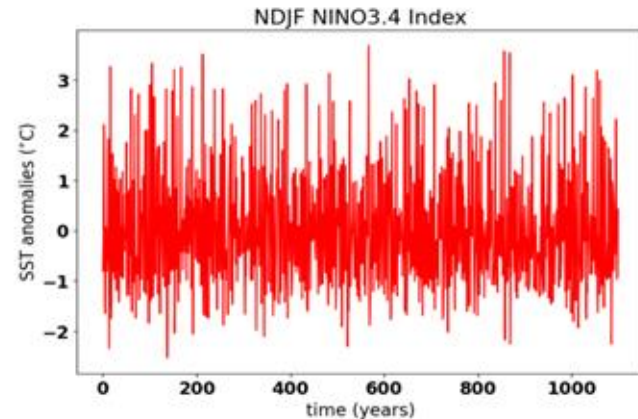
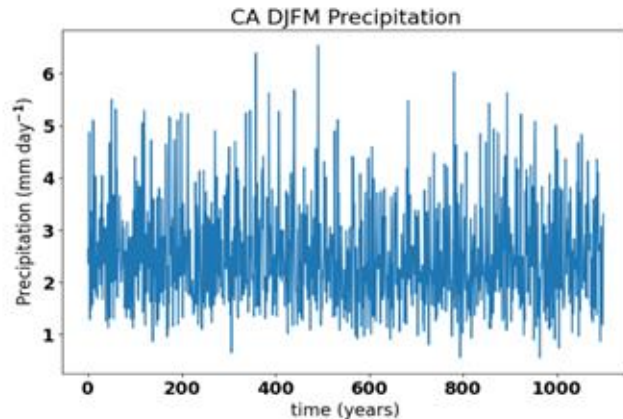
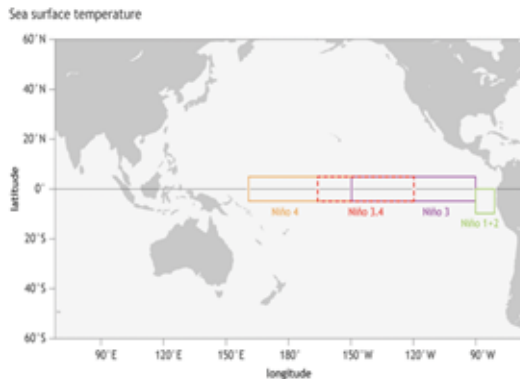


Example: NINO3.4 index in GCM simulations

Goal: Predict precipitation in California from SST anomalies over the NINO3.4 region

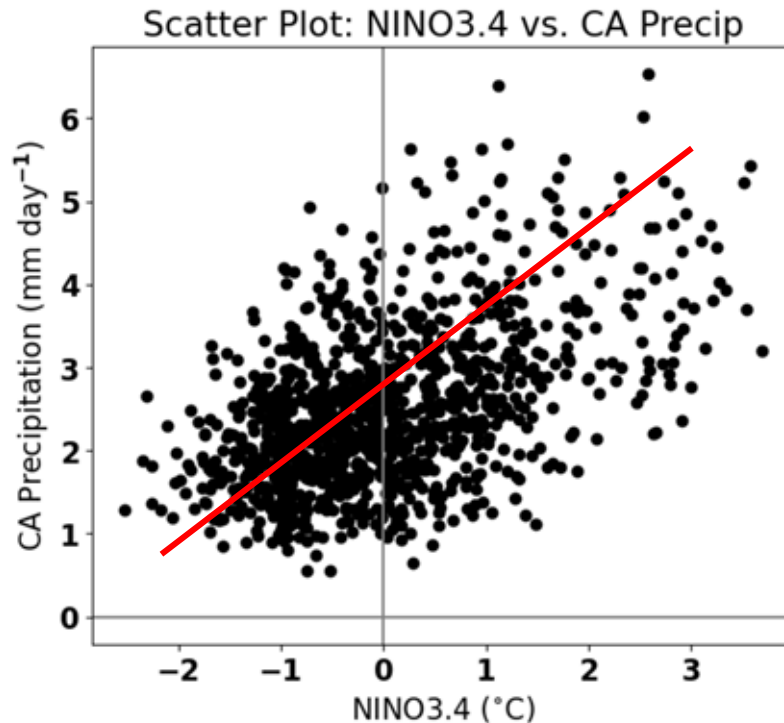
ENSO (El Niño Southern Oscillation) is a recurring global climate pattern involving changes in the temperature of waters in the tropical Pacific Ocean. It is made of different phases:

El Niño represents the warming phase, i.e., California typically gets more winter rainfall and vice versa when there is a **La Niña** event (cooling phase). We have also a neutral phase.



Example: NINO3.4 index

- To what type of ML category linear regression belongs to?
 - Supervised Learning
 - Our dataset is supervised as we are using the precipitation variable as the ground truth
- What type of learning task are we addressing?
 - Regression, as the target is represented by numerical values (not categories)



Linear Regression: Model representation

$$\hat{y} = h_{\theta}(x) = \theta_0 + \theta_1 x$$

- m = number of training example
- n = number of features
- x 's = "input" variable / features
- y 's = "output" variable / "target" variable

	$\theta_0 * 1$	$\theta_1 * x$	y
	intercept	SST anomalies	Precipitation
0	1.0	-0.801642	2.636589
1	1.0	-0.251164	2.354062
2	1.0	2.105125	2.684340
3	1.0	1.958818	4.873932
4	1.0	-1.493576	1.281171
...	...	...	...
1095	1.0	2.220610	3.066943
1096	1.0	-0.275536	2.671779
1097	1.0	-0.965257	1.555862
1098	1.0	-0.883339	1.180074
1099	1.0	0.423453	3.307937

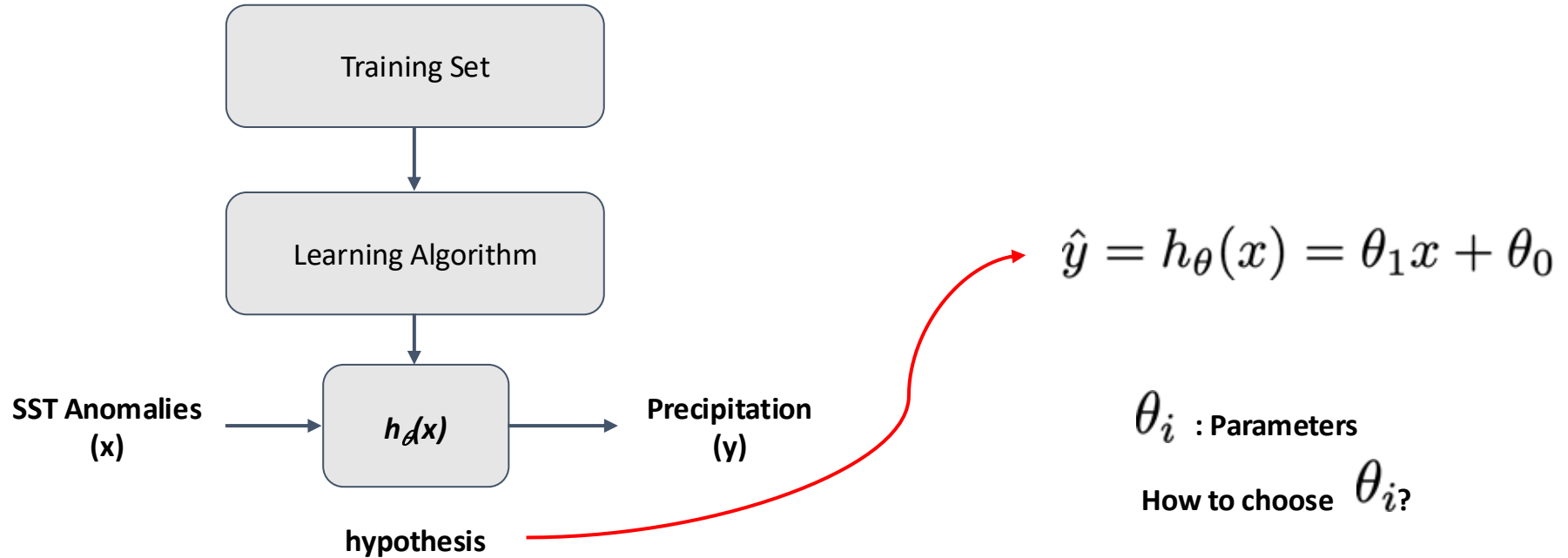
$m = 1100$

$n = 2$



LEAP

Training/Learning Task Overview



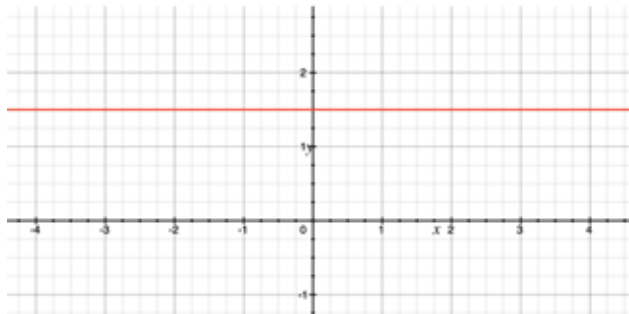
Regression

Linear regression, cost function, gradient descent, and the bias-variance trade-off

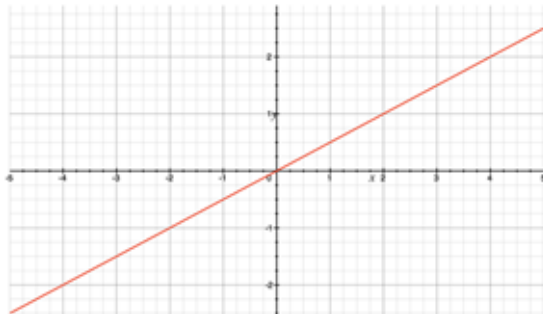


Different parameters identify different models

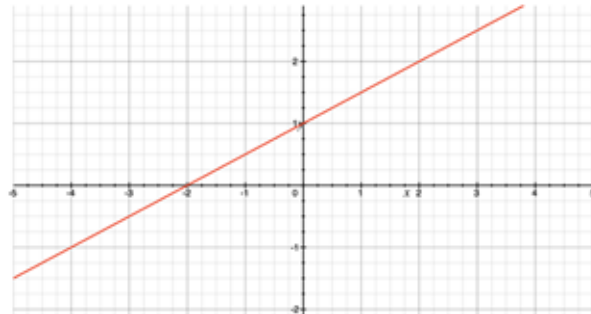
$$\hat{y} = h_{\theta}(x) = \theta_1 x + \theta_0$$



$$h_{\theta}(x) = 0x + 1.5$$



$$h_{\theta}(x) = 0.5x$$



$$h_{\theta}(x) = 0.5x + 1$$

- There are infinite ways to select those parameters. One can randomly pick them, or just impose them, but how do we ensure that they are good parameters?
- From an infinite set of solutions, we are interested in the best one. But what does it mean best?

Cost Function: Mean Squared Error

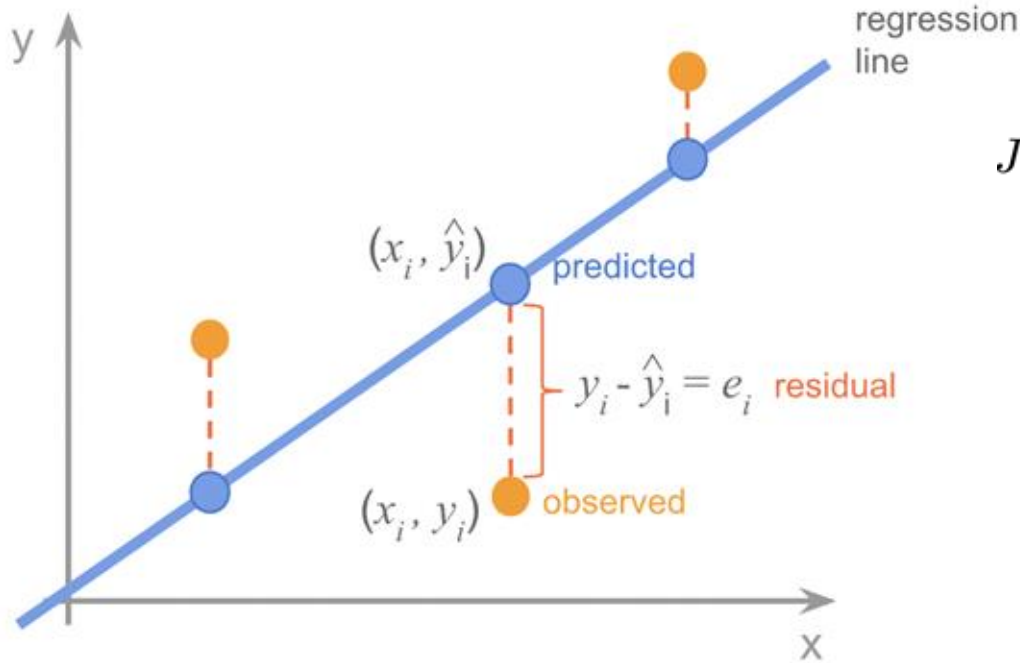
How do we measure how well our model fits the data?

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- $y^{(i)}$ is the actual (true) value for sample i
- $\hat{y}^{(i)} = h_{\theta}(x^{(i)})$ is the predicted value from our model (hypothesis)
- m is the number of training examples
- The cost function measures the average squared difference between predictions and reality
- Squaring penalizes large errors more heavily than small ones
- Our goal: find weights $J(\theta_0, \theta_1)$



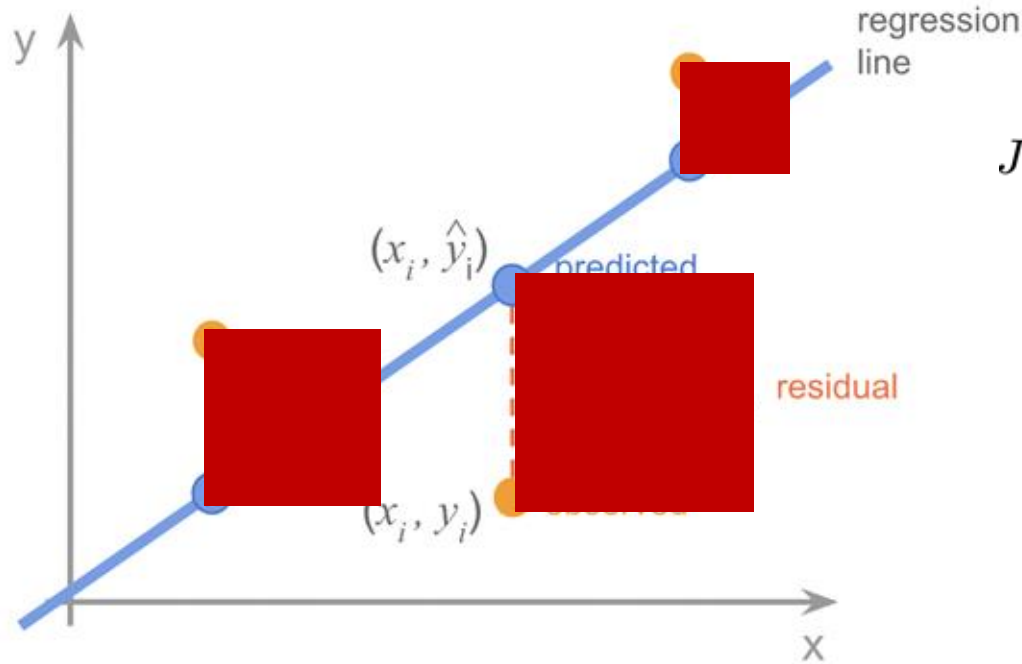
Cost Function: Intuition



$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\hat{y}^{(i)} = h_{\theta}(x^{(i)})$$

Cost Function: Intuition (cont'd)

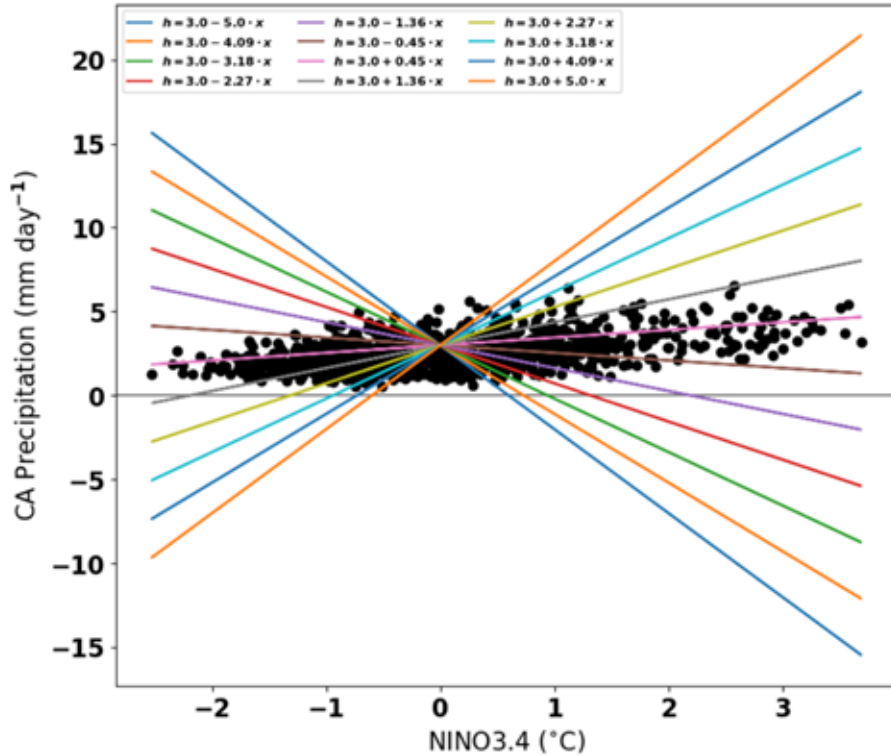


$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\hat{y}^{(i)} = h_{\theta}(x^{(i)})$$

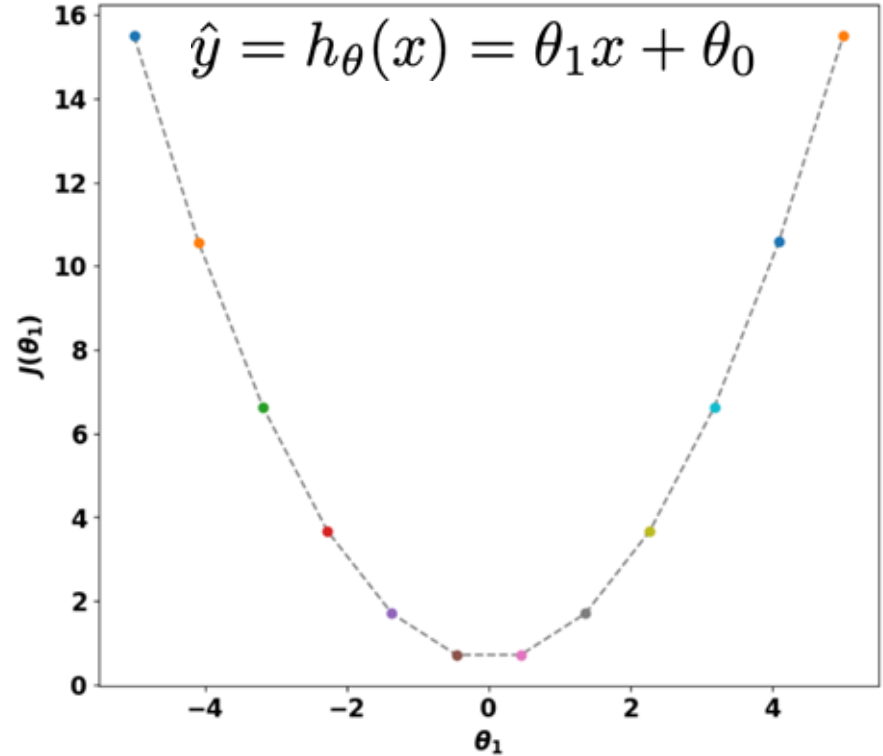
Cost Function Representation: 1D case

Feature Space



Parameter Space

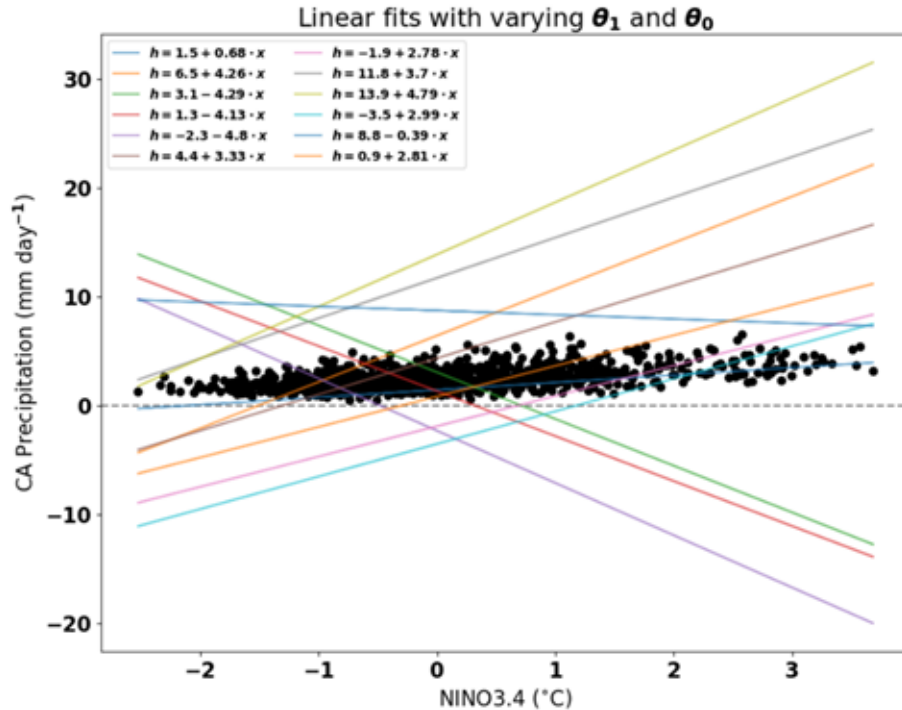
$\theta_0 = 3.0$



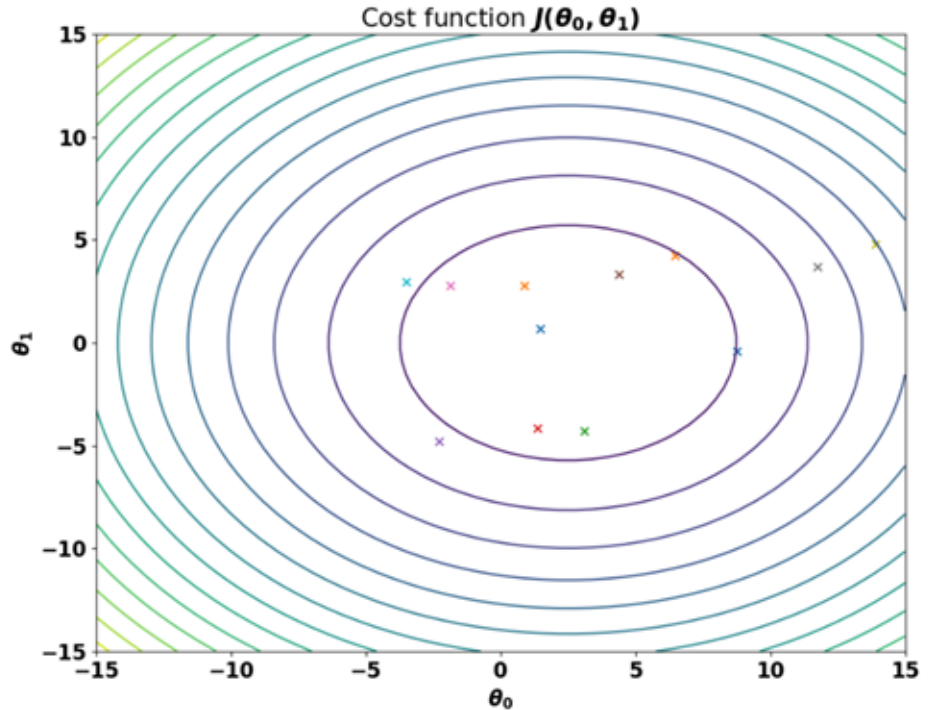
LEAP

Cost Function Representation: 2D case

Feature Space

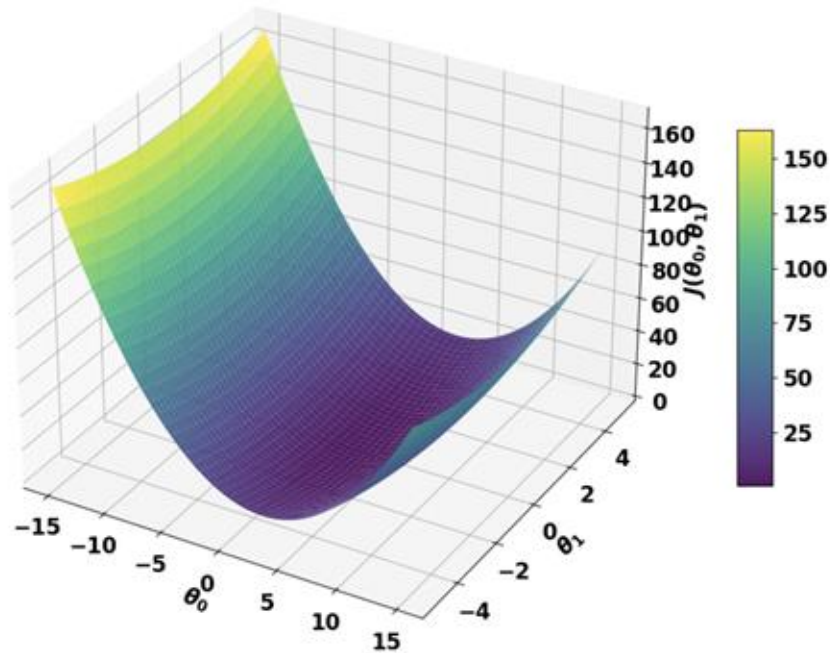


Parameter Space



Cost Function Representation: 3D case

3D Surface of the Cost Function



Regression

Linear regression, cost function, gradient descent, and the bias-variance trade-off



Minimizing the Cost Function

- Have some cost function $J(\theta_0, \theta_1)$
- Want to $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$
- Steps:
 - Start with some θ_0, θ_1
 - Keep updating θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$
 - Hopefully, after some iterations we end up at a minimum of the cost $\rightarrow$ optimal params

... but in which way should we keep updating the parameters?



Gradient Descent: Intuition

Imagine standing on a hilly landscape in fog. You want to reach the lowest valley.

- **Step 1: Feel the slope under your feet (compute gradient)**
 - The gradient tells you which direction is uphill
- **Step 2: Take a step downhill (update weights)**
 - Move in the opposite direction of the gradient
- **Step 3: Repeat until you reach a valley (convergence)**
 - When the gradient is near zero, you've found a minimum

Challenges

- Local minima: for non-convex functions, you might get stuck
- Saddle points: flat regions where gradient is near zero but not a minimum



Gradient Descent

An iterative optimization algorithm to find the minimum of the cost function.

Update rule

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_j}$$

Update the previous value with the new

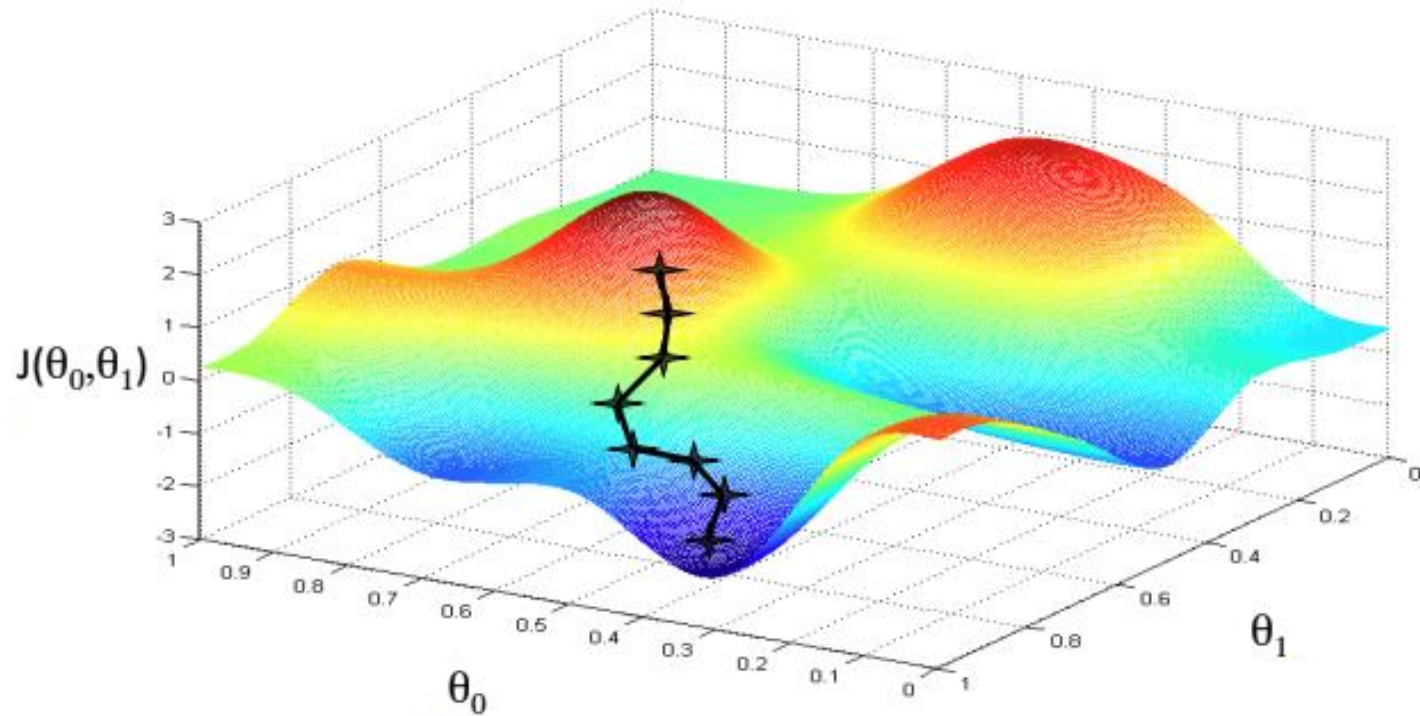
Learning Rate

Partial derivative of the Cost with respect to parameter θ_j

- α = learning rate controls the step size, typically a small number (e.g., 0.001)
- $\partial J(\theta)/\partial \theta_j$ = gradient: points in the direction of steepest ascent
- We move in the opposite direction (steepest descent) to reduce the cost
- Repeat until convergence (cost stops decreasing significantly)

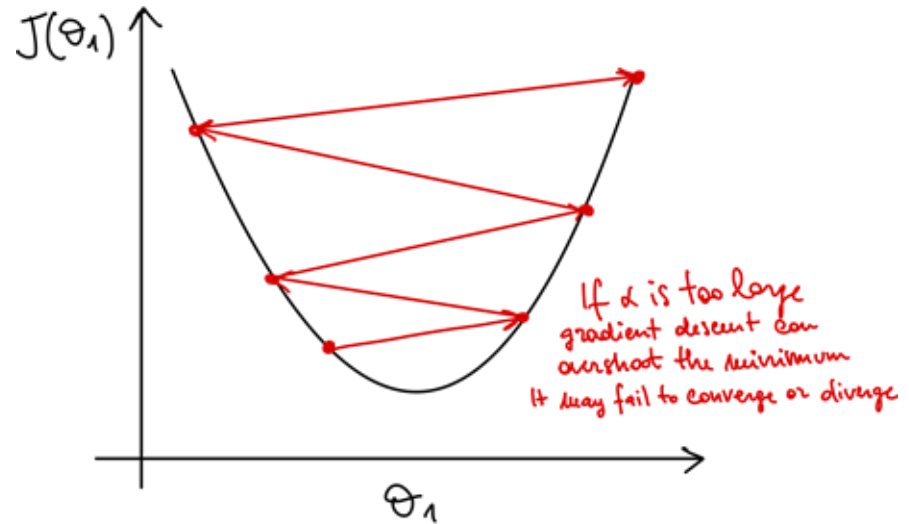
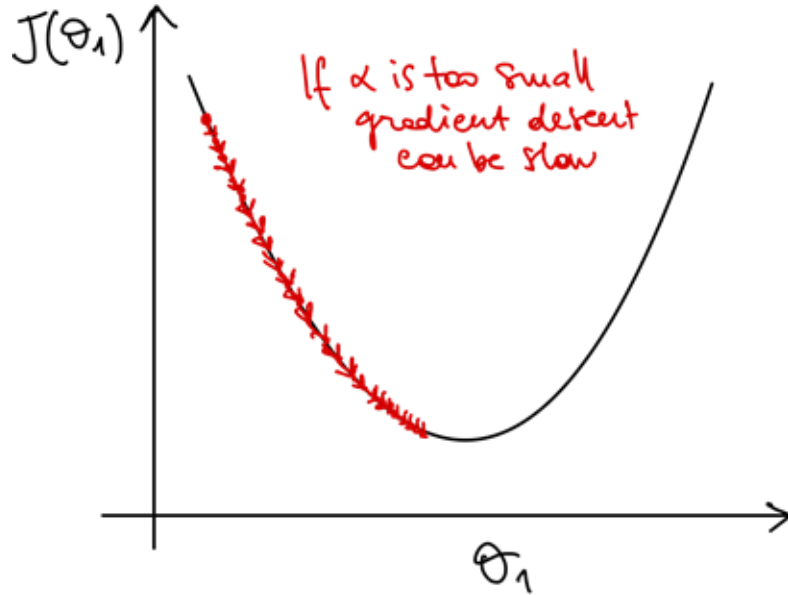


Cost Function Representation: 3D case



How to choose the learning rate

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_j}$$

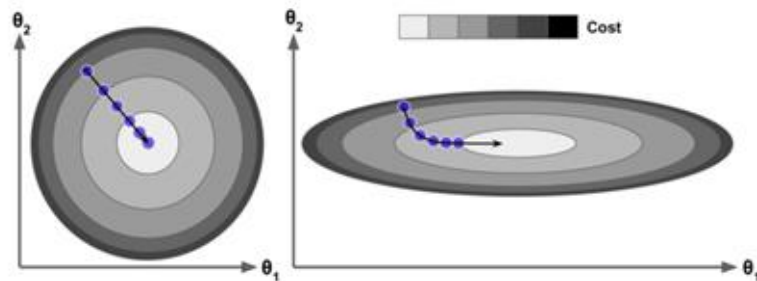


The learning rate can be fixed to a small (positive) number. Even fixed, Gradient Descent will automatically take small steps as we approach a local minimum. So, no need to decrease the learning rate over time (that's not always true).



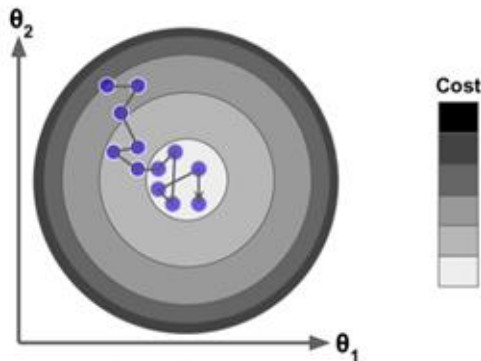
Gradient Descent: Pitfalls

- Selecting the appropriate learning rate is often a trial-and-error process.
- When features vary widely in magnitude, gradient descent may struggle to converge efficiently.
- To address this, it's helpful to scale features to a similar range, typically between 0 and 1.
 - Min-Max Normalization: Rescales features to a fixed range, usually $[0, 1]$.
 - Standardization (Z-score Normalization): Transforms features to have zero mean and unit variance.



Stochastic Gradient Descent

- **Batch Gradient Descent:** Computes the gradient using the entire training set at each step. Requires a lot of memory!!!
- **Stochastic Gradient Descent:** Uses one random instance per step, requiring less memory.
 - The randomness makes the cost function fluctuate, though it decreases on average.
 - Final parameters are typically close to optimal, but not exact due to the noise.
 - This "bouncing" can help escape local minima in non-convex cost functions.
- Mitigation: gradually decrease the learning rate using a learning schedule.



Mini-batch Gradient Descent

- A compromise between Batch (entire dataset) and Stochastic (single instance) gradient descent.
- Computes gradients using small random subsets of the training data, called mini-batches (e.g., 32, 64, 128 instances).
- Offers a good balance between computation efficiency and convergence stability.
- Smoother convergence, but the reduced noise can make it harder to escape local minima or flat regions
- Often used in practice due to its efficiency on large datasets and compatibility with GPU acceleration.

	SST anomalies	Precipitation
0	-0.801642	2.636589
1	-0.251164	2.354062
2	2.105125	2.684340
3	1.958818	4.873932
4	-1.493576	1.281171
...	...	...
1095	2.220610	3.066943
1096	-0.275536	2.671779
1097	-0.965257	1.555862
1098	-0.883339	1.180074
1099	0.423453	3.307937

1100 rows x 2 columns



Polynomial Regression

Extend linear regression by adding polynomial features.

$$\hat{y} = h_{\theta}(x) = \theta_1 SST_{anomaly} + \theta_2 SST_{anomaly}^2 + \theta_0$$

- Still a linear model in terms of weights (linear in parameters)
- Can capture non-linear relationships in data
- Higher degree = more flexible model, but **higher risk of overfitting**
- Key question: how do we choose the right polynomial degree?

This leads us to the bias-variance trade-off...



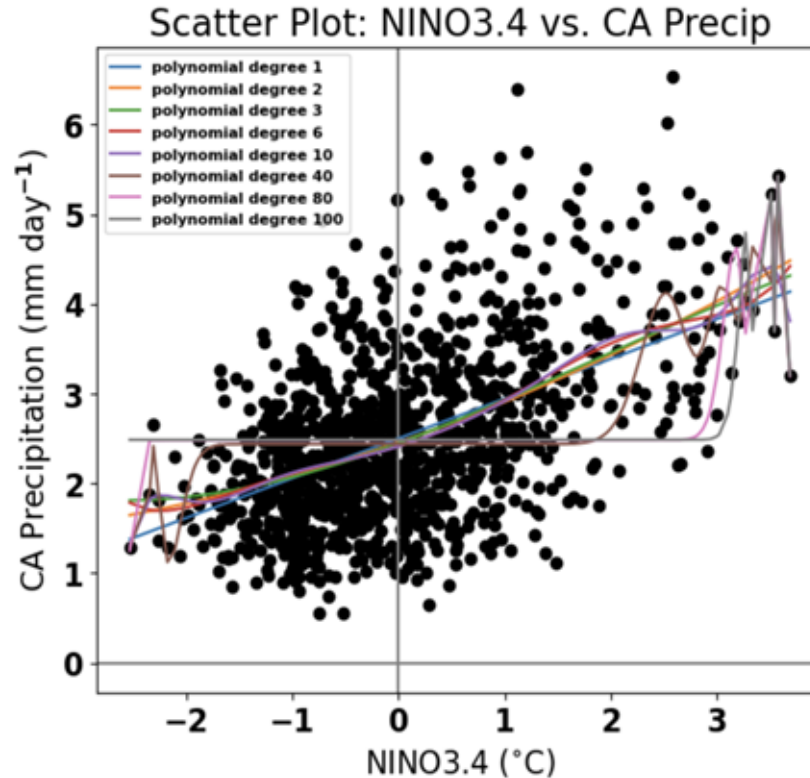
Regression

Linear regression, cost function, gradient descent, and the bias-variance trade-off



Overfitting: How to Choose the Right Polynomial Degree?

The polynomial degree 1 model (standard Linear Regression) is probably too simple



Polynomial models of degree higher than 10 are probably too complex and may overfit training data

Bias-Variance Trade-off

Underfitting

High bias, low variance

Model is too simple.
Cannot capture the true pattern.
Bad on both training and test data.

Example: fitting a line to clearly curved data

Solution: Add more parameters in the model (e.g., consider additional polynomial features)

Just Right

Balanced

Model captures the underlying pattern without memorizing noise.
Good on both training and test data.

This is the sweet spot we aim for.

Overfitting

Low bias, high variance

Model is too complex.
Memorizes training data including noise.
Great on training, poor on unseen data.

Example: degree-20 polynomial for 10 data points

Solution: Reduce the model complexity, e.g., reduce the parameters of the model, add more data, use regularization techniques (see next)

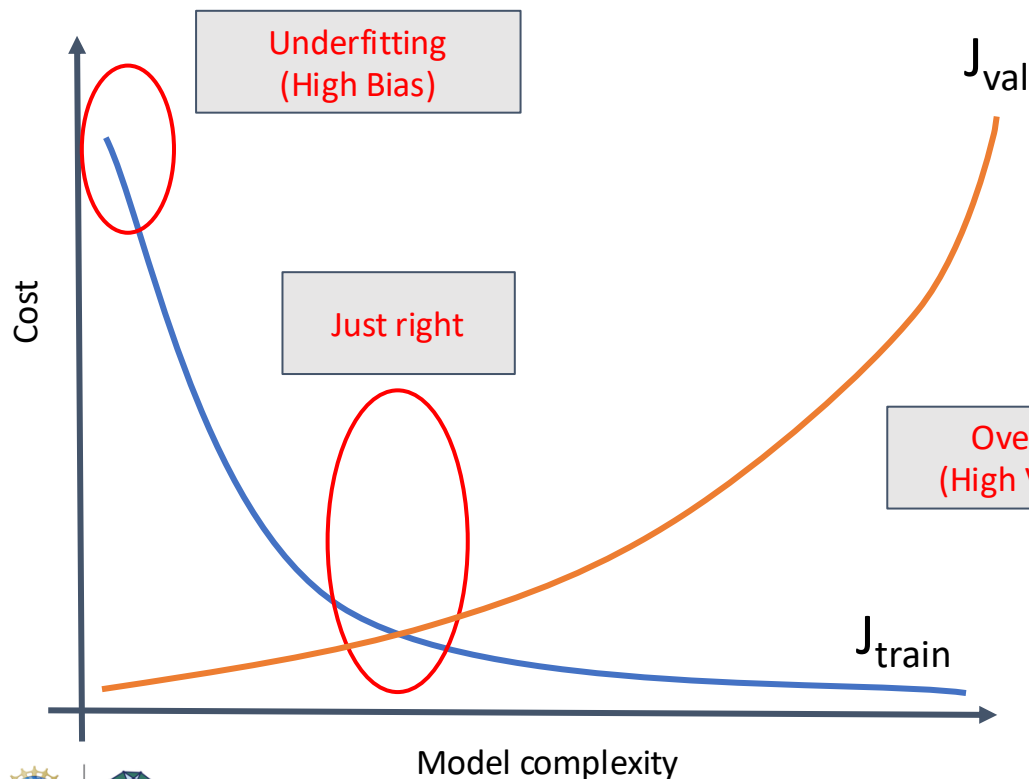
What should you do in practice, and how can you understand which case you are in?

Take some validation examples apart (not seen during training), and plot learning curves



LEAP

Diagnose Bias vs. Variance



High Bias (Underfitting)

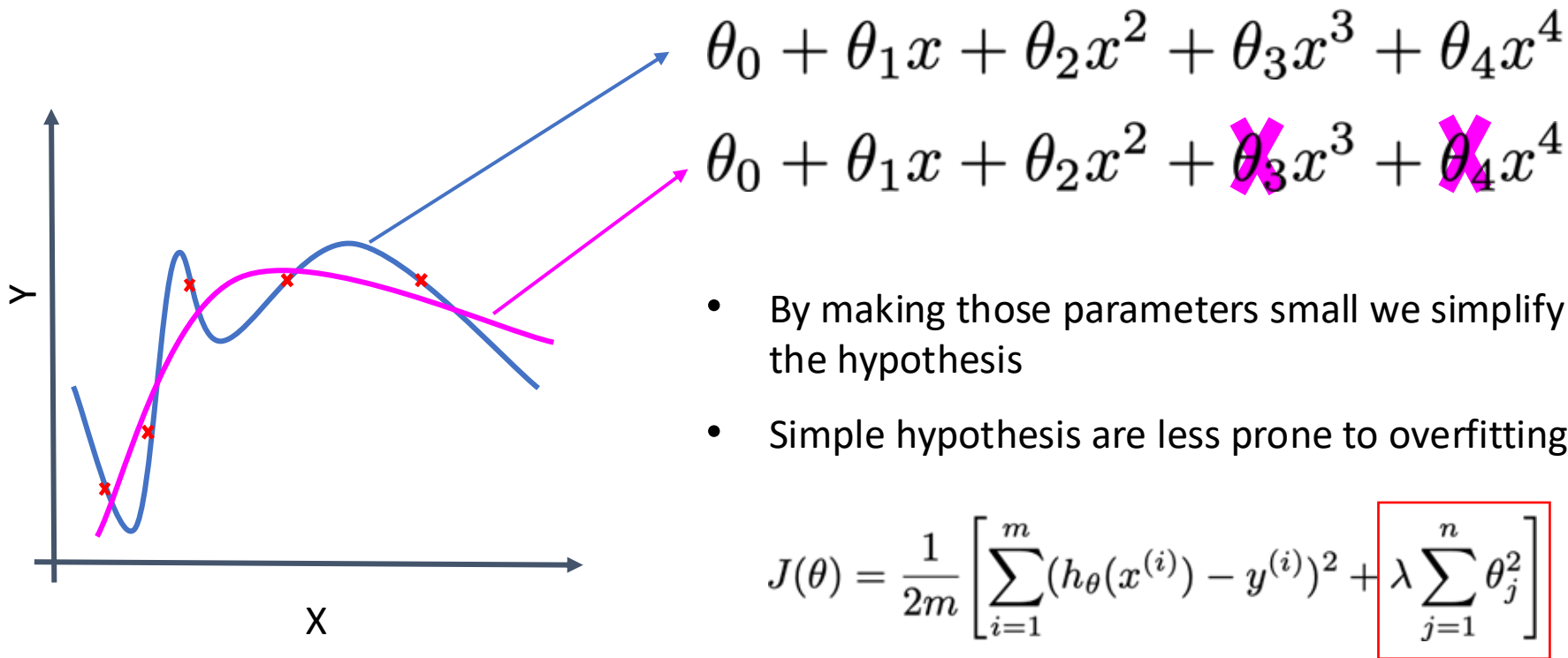
- J_{train} will be high
- $J_{\text{val}} \sim J_{\text{train}}$

High Variance (Overfitting)

- J_{train} will be low
- $J_{\text{val}} \gg J_{\text{train}}$



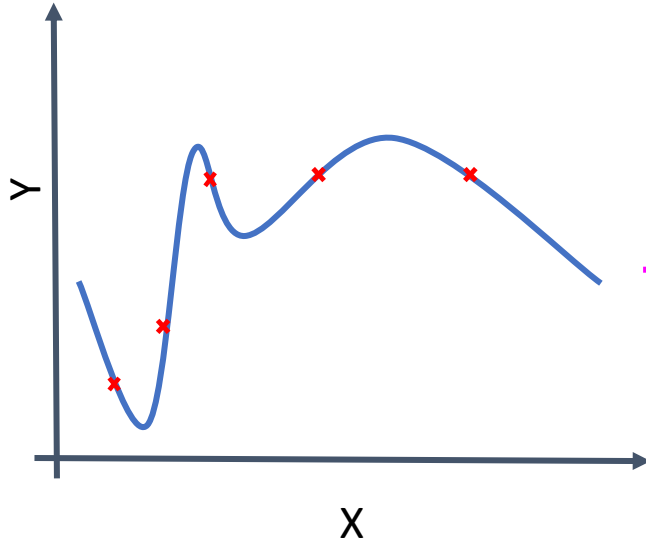
Regularization: intuition



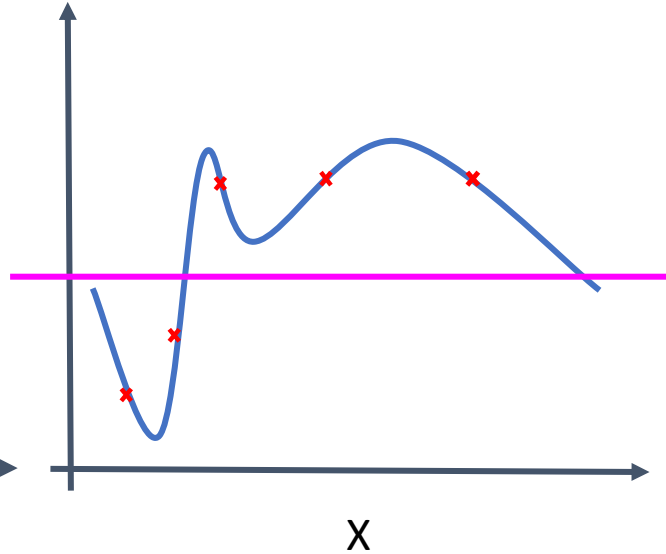
Regularization: How to choose λ

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

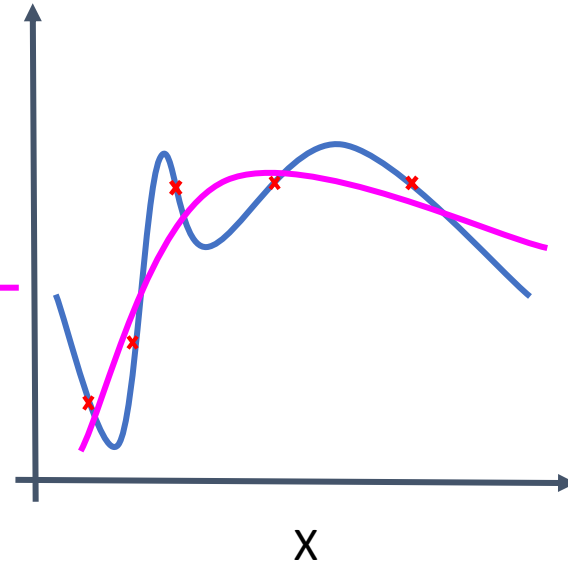
$\lambda = 0$



$\lambda \gg 0$



λ just right



LEAP

Artificial Neural Networks

Architecture, activation functions, cost functions, and backpropagation

Biological Inspiration

- Brain function occurs through the firing of neurons
- Neurons connect through synapses that propagate electrical impulses
- ~100 billion neurons and ~100 trillion synapses in the human brain

Artificial Neural Networks (ANNs)

- Algorithms inspired by (but much simpler than) the brain
- First wave in the 1980s-90s, then a resurgence with deep learning (2012+)
- Now state-of-the-art for many tasks in computer vision, NLP, and science



The Perceptron: Simplest Neural Unit

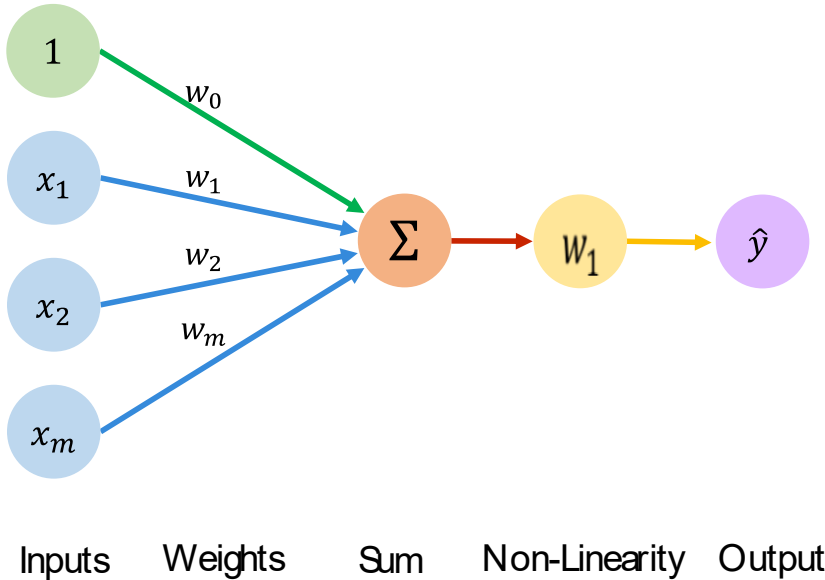
A single neuron that takes weighted inputs, sums them, and applies an activation function.

$$\text{output} = g(w_1x_1 + w_2x_2 + \dots + w_nx_n + w_0)$$

- **Inputs ($x_1 \dots x_n$):**
 - Feature values fed into the neuron
- **Weights ($w_1 \dots w_n$):**
 - Learned parameters that scale each input's importance
- **Bias (w_0):**
 - An offset term (like the intercept in linear regression)
- **Activation function $g()$:**
 - Non-linear transformation applied to the weighted sum



The Perceptron: Forward Propagation



Output

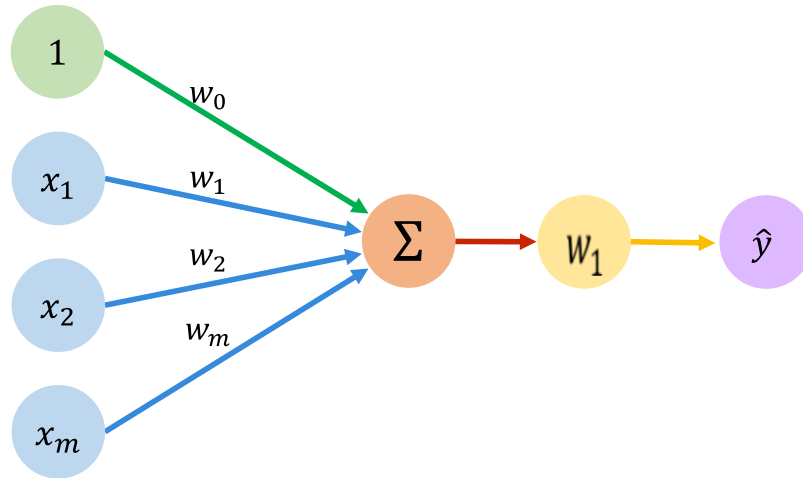
Linear combination of inputs

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Bias

The Perceptron: Forward Propagation



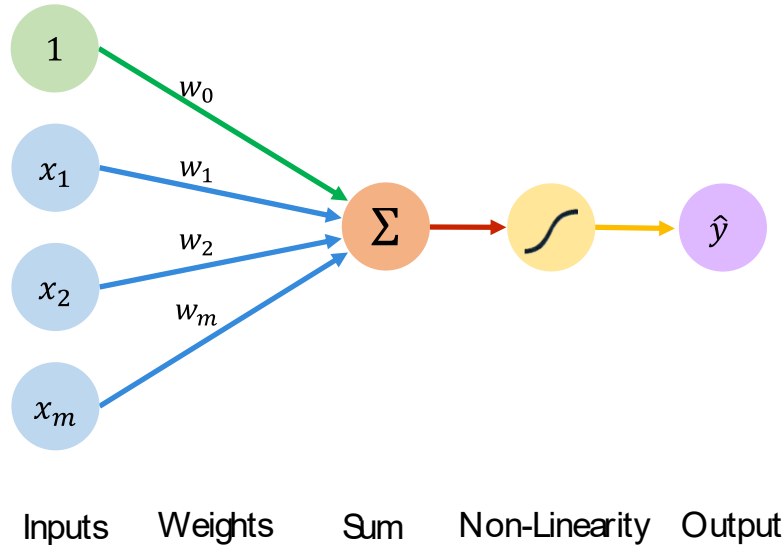
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

$$\hat{y} = g (w_0 + \mathbf{X}^T \mathbf{W})$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Inputs Weights Sum Non-Linearity Output

The Perceptron: Forward Propagation

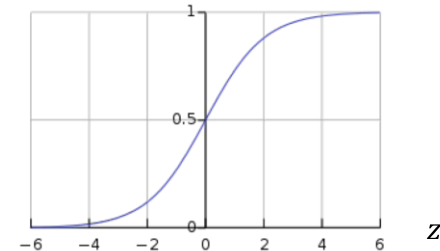


Activation Functions

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

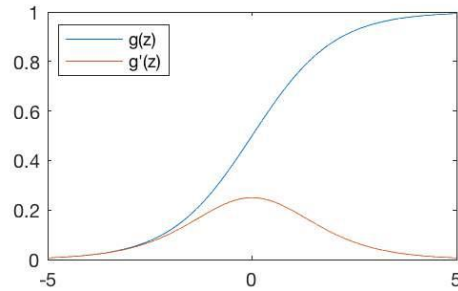
- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Common Activation Functions

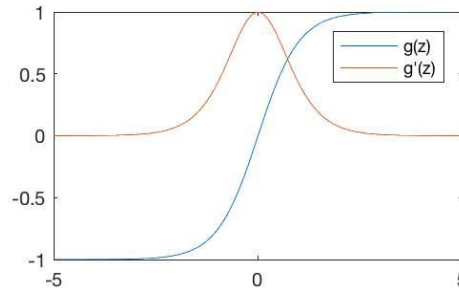
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

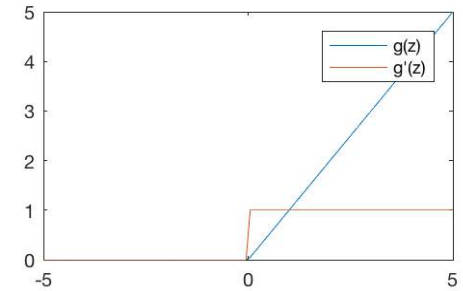
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



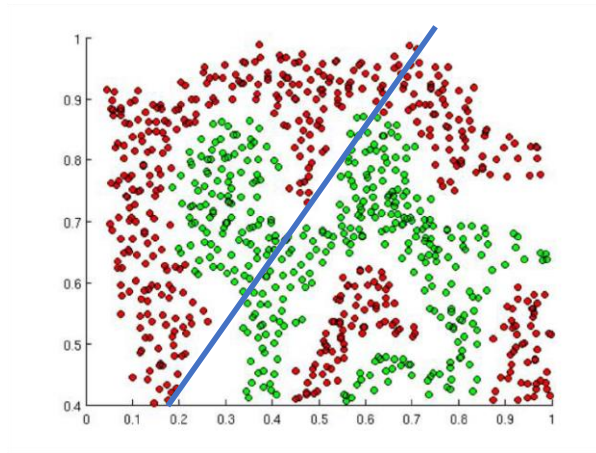
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

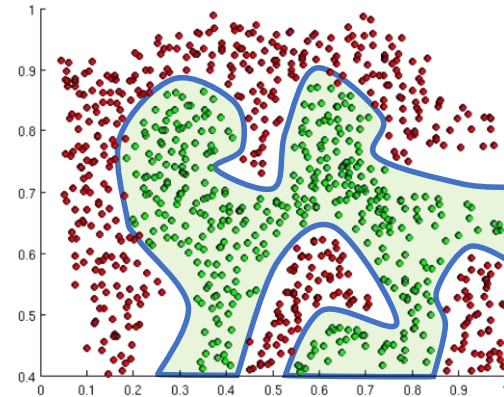
NOTE: All activation functions are non-linear

Importance of Activation Functions

The purpose of activation functions is to introduce non-linearities into the network

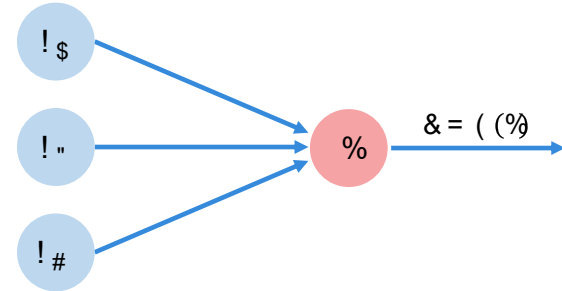
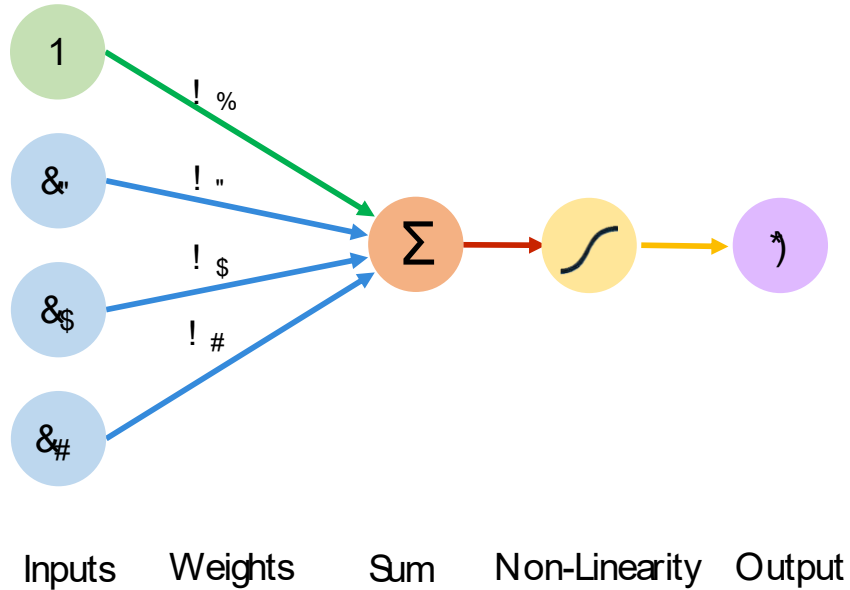


Linear Activation functions produce linear decisions no matter the network size



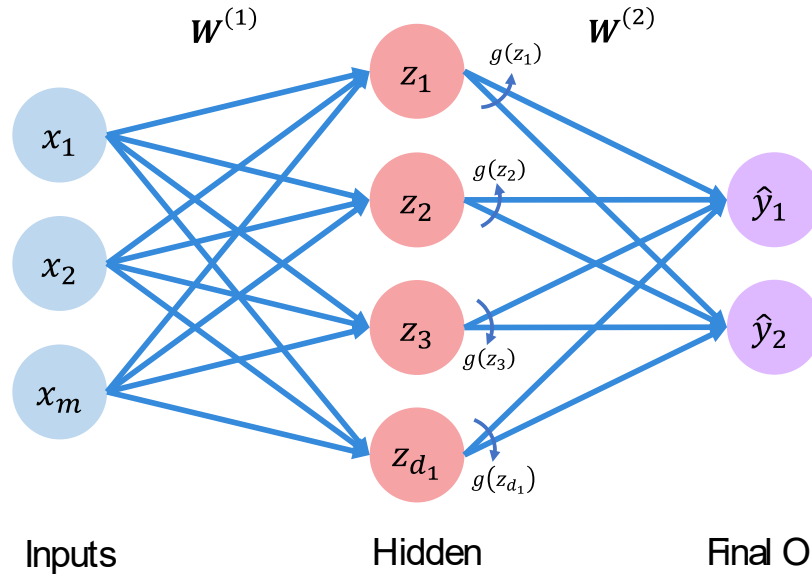
Non-linearities allow us to approximate arbitrarily complex functions

The Perceptron: Simplified



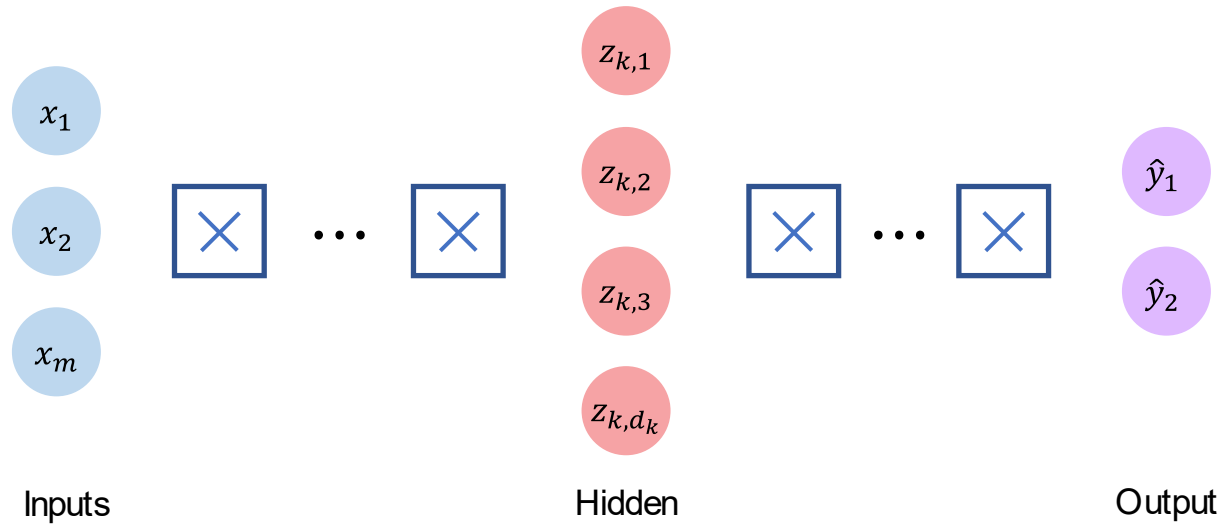
$$\% =) * + , \# ! -) - . \$$$

Single Layer Neural Network



$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)} \right)$$

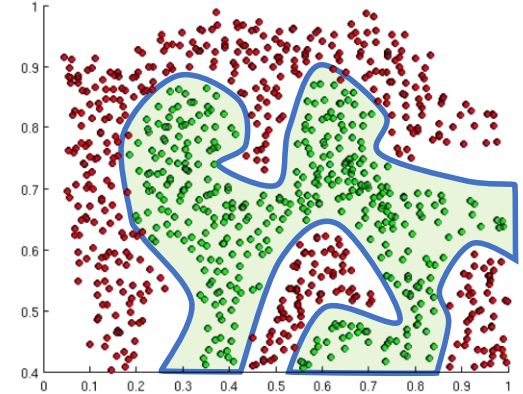
Deep Neural Network



$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{d_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

The Role of Non-Linearity

- **Without activation functions:**
 - Multiple linear layers collapse into a single linear transformation: $f(g(x)) = Ax + b$
 - No matter how many layers, the network can only learn linear decision boundaries
- **With activation functions:**
 - Each layer can learn increasingly complex non-linear features
 - Universal Approximation Theorem: a single hidden layer with enough neurons can approximate any continuous function
 - Deeper networks learn hierarchical representations more efficiently



Non-linearities allow us to approximate arbitrarily complex functions

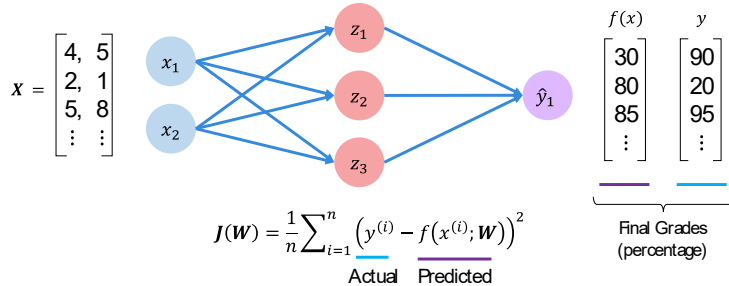
Cost Functions for Neural Networks

Regression

Most common for continuous outputs.
Penalizes large errors quadratically.

Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers

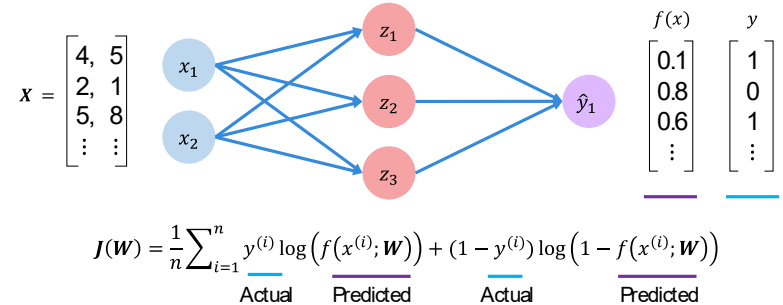


Classification

Cross-entropy for probability outputs.
Binary CE for 2 classes, Categorical CE for multi-class.
Paired with sigmoid (binary) or softmax (multi-class).

Binary Cross Entropy Loss

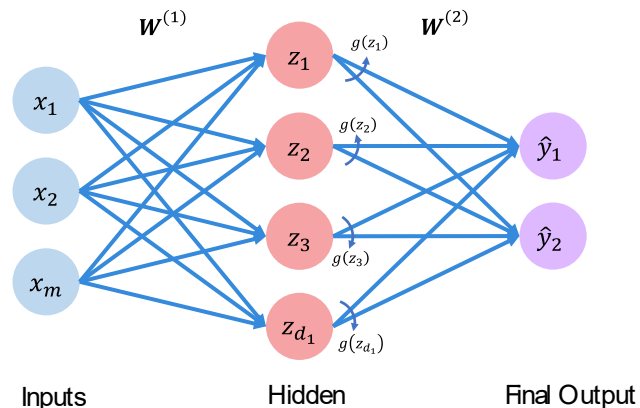
Cross entropy loss can be used with models that output a probability between 0 and 1



How ANNs Learn: Forward Pass

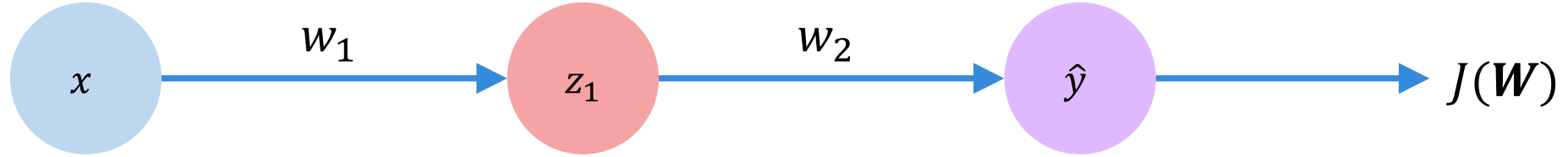
Data flows forward through the network to produce a prediction.

- **Step 1: Input features are fed to the input layer**
- **Step 2: For each hidden layer:**
 - Compute $z = Wx + b$ (weighted sum)
 - Apply activation: $g(z)$
- **Step 3: Output layer produces final prediction $\hat{y}$**
- **Step 4: Compute loss: $L = J(y, \hat{y})$**
 - Compare prediction to true label and quantify the error



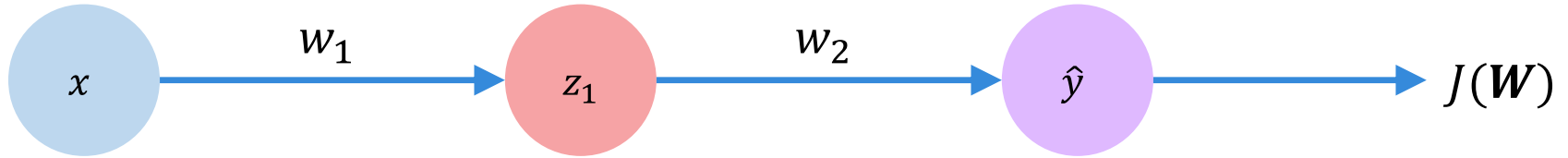
Now, how do we use this error to update the weights?

How NNs Learn: Compute gradients



How does a small change in one weight (ex. w_2) affect the final loss $J(W)$?

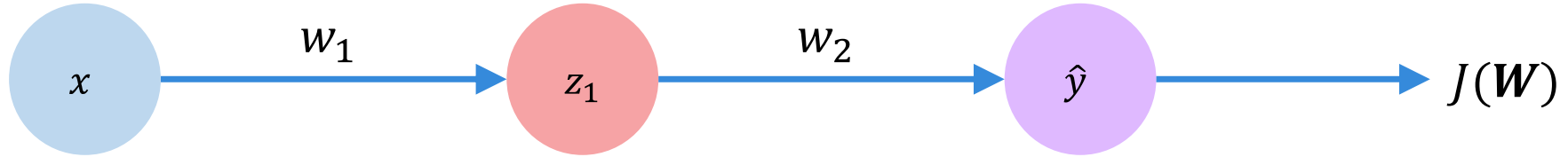
How NNs Learn: Compute gradients



$$\frac{\partial J(\mathbf{W})}{\partial w_2} =$$

Let's use the chain rule!

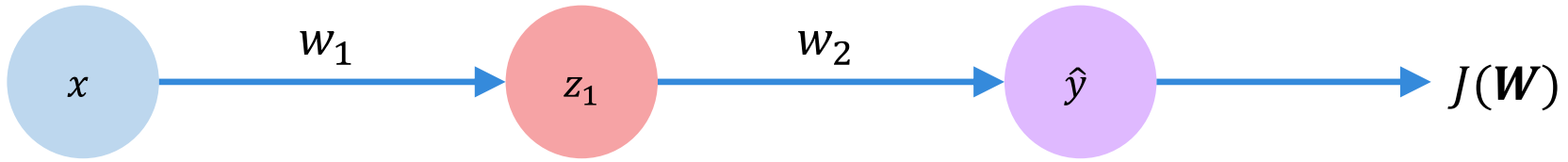
How NNs Learn: Backpropagation algorithm



$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$



How NNs Learn: Backpropagation algorithm

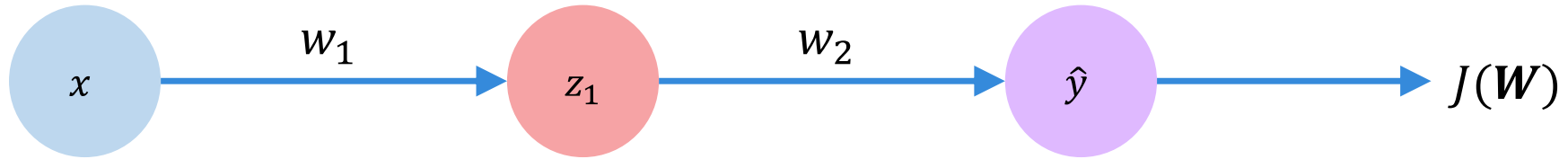


$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

Apply chain rule!

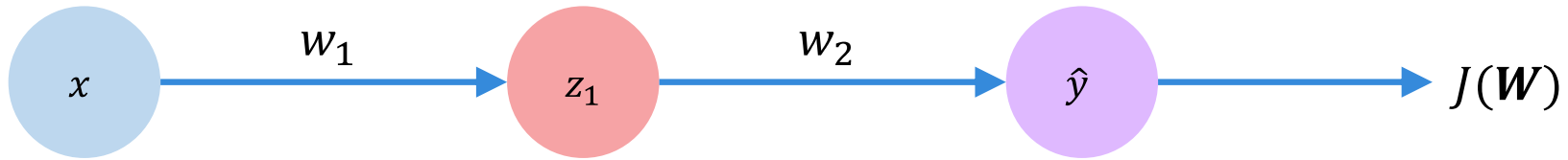
Apply chain rule!

How NNs Learn: Backpropagation algorithm



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple bar}} * \frac{\partial \hat{y}}{\partial z_1} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue bar}}$$

How NNs Learn: Backpropagation algorithm



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple bar}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{blue bar}} * \frac{\partial z_1}{\partial w_1}$$

Repeat this for every weight in the network using gradients from later layers

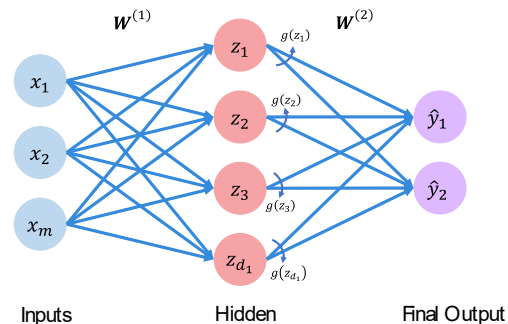
Gradient Descent for ANNs

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Forward pass: compute predictions and loss
3. Backward pass: propagate error gradients from output to input

Apply the chain rule

Each layer passes its gradient to the previous layer, store gradients $\frac{\partial J(W)}{\partial W}$

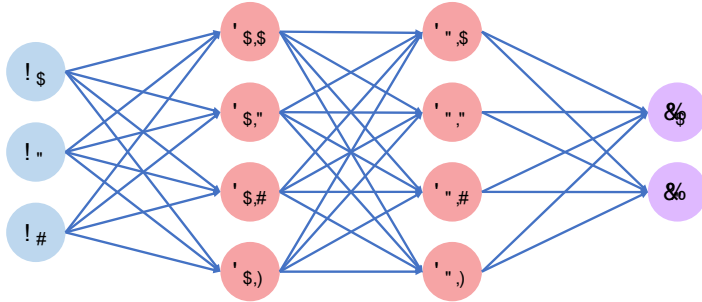
3. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
4. Repeat for many iterations (epochs) over the dataset
5. Monitor how training and validation loss look across epochs



Overfitting and Regularization

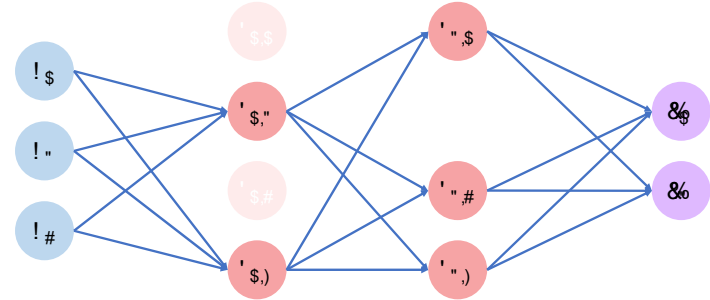
Regularization 1: Dropout

- During training, randomly set some activations to 0



Regularization 1: Dropout

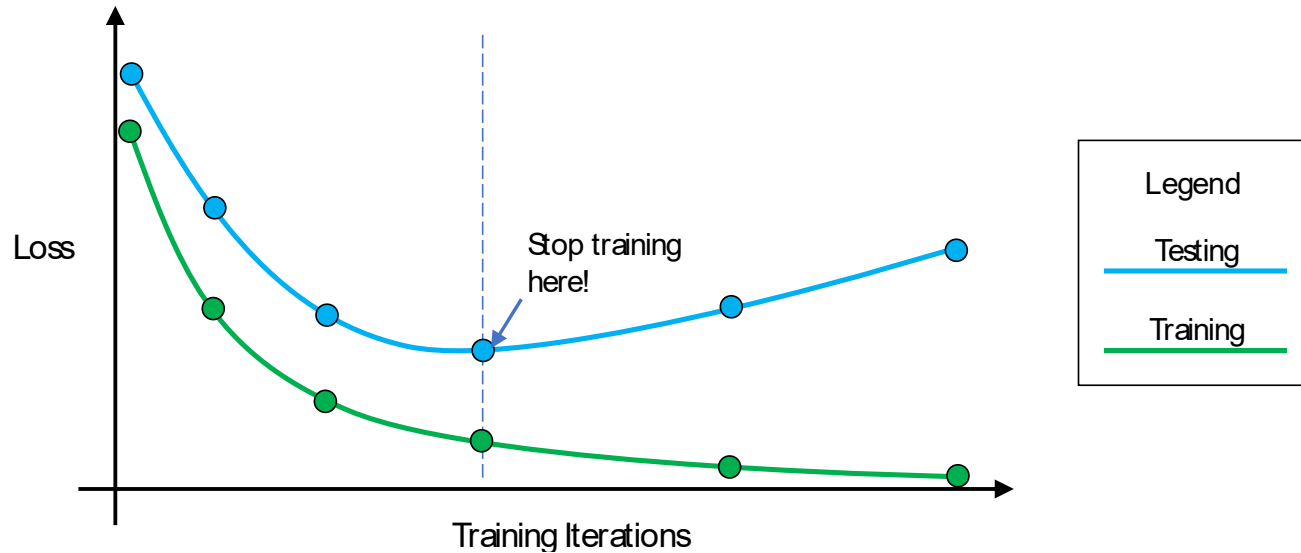
- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node



Ref: Srivastava et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR.

Regularization 2: Early Stopping

- Stop training before we have a chance to overfit



PyTorch Framework

Building and training neural networks in practice

PyTorch: Overview

An open-source deep learning framework developed by Meta (Facebook) AI Research.

- **Dynamic computation graphs:**
 - Build the graph on-the-fly (eager execution). Easy to debug with standard Python tools
- **Automatic differentiation (autograd):**
 - Automatically computes gradients for backpropagation. No manual calculus needed!
- **GPU acceleration:**
 - Seamlessly move tensors and models between CPU and GPU with `.to(device)`
- **Rich ecosystem:**
 - torchvision, torchaudio, torchtext, PyTorch Lightning, Hugging Face integration
- **Pythonic:**
 - Feels like NumPy with GPU support and automatic gradients



Tensors and Autograd

Tensors

- Multi-dimensional arrays (like NumPy ndarrays but with GPU support)
- Operations: **torch.tensor()**, **torch.zeros()**, **torch.randn()**, reshaping, slicing
- Move to GPU: **tensor.to('cuda')** or **tensor.cuda()**

Autograd: automatic differentiation

- Set **requires_grad=True** on tensors you want gradients for
- PyTorch records all operations in a computational graph
- Call **loss.backward()** to compute all gradients automatically
- Access gradients via **tensor.grad**
- Handles arbitrarily complex computation graphs



Dataset Class

torch.utils.data.Dataset

- Custom class that implements `__len__()` and `__getitem__(idx)`
- `__len__`: returns the total number of samples
- `__getitem__`: returns one sample (input, target) given an index
- Handles loading, preprocessing, and augmentation per sample

```
import os
import pandas as pd
from torchvision.io import decode_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = decode_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```



DataLoaders

torch.utils.data.DataLoader

- Wraps a Dataset and provides batching, shuffling, and parallel loading
- Key arguments: batch_size, shuffle, num_workers, pin_memory
- Handles memory-efficient loading of large datasets
- Automatically creates mini-batches for training
- In this **example both training_data and test_data are instances/objects of a Dataset class**

```
from torch.utils.data import DataLoader
```

```
train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)  
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```



Building a Neural Network in PyTorch

Define a model by subclassing torch.nn.Module.

```
class MyANN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(10, 64)
        self.layer2 = nn.Linear(64, 32)
        self.output = nn.Linear(32, 1)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.layer1(x))
        x = self.relu(self.layer2(x))
        return self.output(x)
```



PyTorch Training Loop

```
model = MyANN().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

for epoch in range(num_epochs):
    for batch_x, batch_y in train_loader:
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        pred = model(batch_x)          # Forward pass
        loss = criterion(pred, batch_y) # Compute loss

        optimizer.zero_grad()          # Clear old gradients
        loss.backward()                 # Perform Backpropagation
        optimizer.step()                # Update weights
```

5 key steps per iteration: forward pass → loss → zero_grad → backward → step



Optimizers, Callbacks, and Device Handling

- **Common optimizers:**

- torch.optim.SGD, Adam, AdamW, RMSprop, LBFGS

$$\mathbf{W} \leftarrow \mathbf{W} - \boxed{\eta} \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

- **Learning rate schedulers (callbacks):**

- StepLR: decay lr every N epochs
- ReduceLROnPlateau: reduce lr when validation loss plateaus
- CosineAnnealingLR: smooth cyclic lr schedule

- **Device handling:**

- device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
- model.to(device) and data.to(device) move computation to GPU
- Multi-GPU: torch.nn.DataParallel or DistributedDataParallel



Day 1: Key Takeaways

- ML learns patterns from data to generalize to unseen examples
- Supervised learning (regression + classification) requires labeled data (ground truth)
- Linear/polynomial regression minimize MSE using gradient descent
- Bias-variance tradeoff: balance model complexity vs. generalization
- Regularization prevents overfitting by reducing model complexity
- Neural networks stack layers of non-linear transformations
- Backpropagation efficiently computes gradients via the chain rule
- PyTorch provides tensors + autograd for flexible model building

